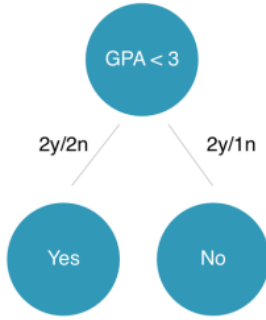


ADABOOST ML ALGORİTMASI

Adaptive Boosting, Zayıf öğrencileri (weak learner) bir araya getirerek güçlü bir model (strong learner) oluşturmayı amaçlayan bir **boosting** algoritmasıdır.

Genellikle karar ağaçlarının *decision stump* (derinliği 1 olan karar ağaçları) versiyonlarını kullanır ve bu modellerin hatalarına göre ağırlıklandırma yapar.

bu işte *decision stump* örneği mesela.



Temel prensibi;

1. İlk olarak, tüm veri setine eşit ağırlık verir.
2. İlk zayıf öğrenci (örneğin, basit bir karar ağacı) modeli oluşturur ve veri setindeki örnekleri sınıflandırmayı dener.
3. Modelin **yanlış sınıflandırdığı** örnekler daha fazla ağırlık verirken, doğru sınıflandırdıklarının ağırlığını azaltır.
4. Daha sonra yeni bir zayıf öğrenci, bu güncellenmiş ağırlıklara göre eğitilir. Bu, yeni modelin önceki modelin zorlandığı örnekler daha fazla odaklanmasını sağlar.
5. Bu süreç, belirli bir sayıda model oluşturulana veya istenen doğruluk seviyesine ulaşılan kadar tekrarlanır.
6. Son olarak, tüm bu zayıf öğrencilerin tahminleri birleştirilir. Genellikle her birinin performansı (doğruluk oranı) göz önünde bulundurularak ağırlıklı bir oylama yapılır ve nihai tahmin ortaya çıkar.

3 AdaBoost Fonksiyonu

AdaBoost'un çıktısı aşağıdaki gibi hesaplanır:

$$F(x) = \sum_{m=1}^M \alpha_m h_m(x)$$

Burada:

- $h_m(x)$: m . zayıf öğrenci,
- α_m : h_m için ağırlık,
- M : toplam zayıf model sayısıdır.

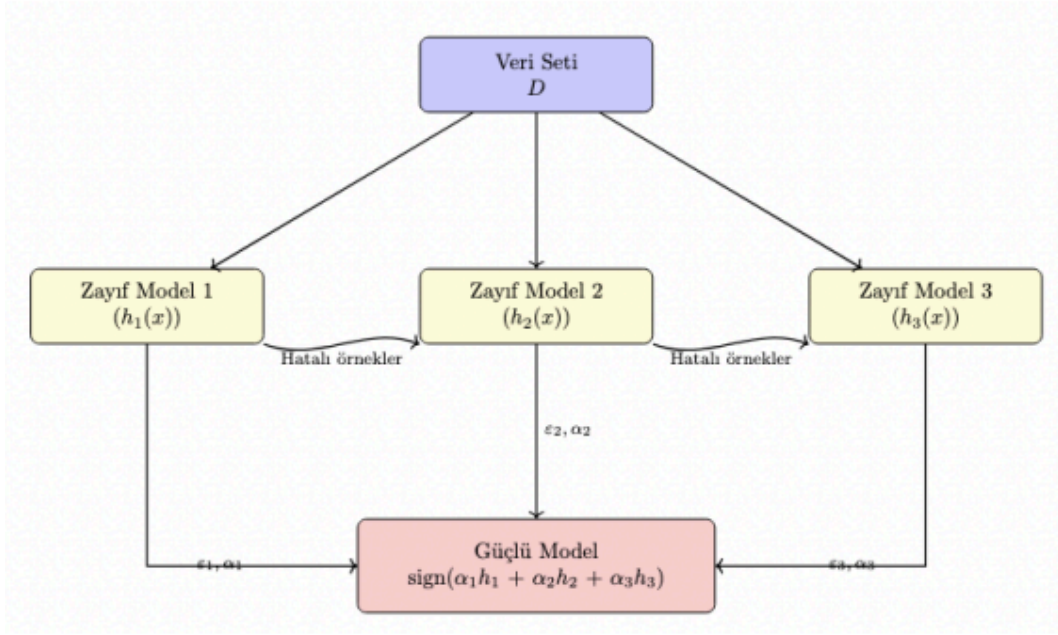


Figure 1: Adaboost Sürecinin Genel Şeması

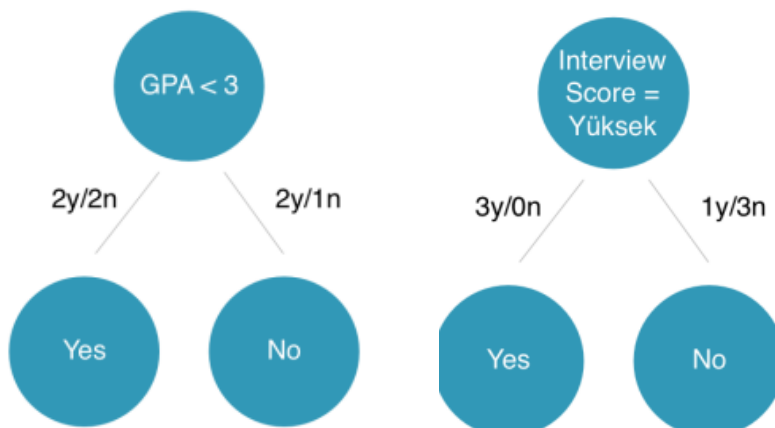
Example;

ID	GPA	Interview Score	Approval (Y)
1	<3.0	Düşük	No
2	<3.0	Yüksek	Yes
3	<3.0	Yüksek	Yes
4	>3.0	Düşük	No
5	>3.0	Yüksek	Yes
6	>3.0	Normal	Yes
7	<3.0	Normal	No

Şimdi yandaki örnek veri seti üzerinden bir örnek çözümim daha iyi otursun.

Table 1: AdaBoost için örnek veri seti

Adım 1- İlk Stump ve Tahminler



Elimizde bu iki stump var ve bilindiği üzere

Seçilen Stump: Interview Score = Yüksek

Tahminler

ID	GPA	Interview Score	Approval (Y)	Tahmin	Tahmin Doğru mu?
1	<3.0	Düşük	No	No	Yes
2	<3.0	Yüksek	Yes	Yes	Yes
3	<3.0	Yüksek	Yes	Yes	Yes
4	>3.0	Düşük	No	No	Yes
5	>3.0	Yüksek	Yes	Yes	Yes
6	>3.0	Normal	Yes	No	No
7	<3.0	Normal	No	No	Yes

Table 2: Sadece ID 6 yanlış tahmin edilecektir

AdaBoost'un yaptığı hataları yeni bir zayıf modele aktarma mantığı, **örneklerin ağırlıklarını (weights) dinamik olarak güncelleme** prensibine dayanır. Bu süreç adım adım şu şekilde işler:

1. **İlk Adım (Başlangıç Ağırlıkları):** Algoritma ilk zayıf öğrenciyi (modeli) eğitmeden önce, veri setindeki tüm eğitim örneklerine eşit ağırlık atar. Yani her örnek aynı derecede önemlidir.
2. **Yanlış Sınıflandırma ve Ağırlık Artışı:** İlk model eğitildikten sonra, veri setindeki örnekleri sınıflandırır. Bu modelin **yanlış sınıflandırdığı** örneklerin ağırlıkları artırılır. Bu artışın mantığı, bir sonraki modelin bu "zor" örnekler daha fazla odaklanmasını sağlamaktır. Yanlış sınıflandırma, örneklerin "daha önemli" hale gelmesine neden olur.
3. **Doğru Sınıflandırma ve Ağırlık Azalışı:** Aynı şekilde, ilk modelin **doğru sınıflandırdığı** örneklerin ağırlıkları azaltılır. Bunun sebebi, bu örneklerin kolayca öğrenilebildiğini ve bir sonraki modelin onlara daha az dikkat etmesi gerektiğini belirtmektir.
4. **Yeni Model Eğitimi:** Ağırlıkları güncellenmiş bu yeni veri seti (aslında veri setinin kendisi değil, ağırlık listesi güncellenmiş hali) ile ikinci zayıf öğrenci eğitilir. Ağırlığı artırılmış olan örnekler, yeni modelin eğitim sürecinde daha belirgin hale gelir ve model, onları doğru sınıflandırmak için daha fazla "çaba sarf eder".
5. **Sürecin Tekrarı:** Bu döngü, her model eğitiminden sonra tekrarlanır. Her yeni model, bir önceki modelin zorlandığı (yani ağırlıkları yüksek olan) örnekler üzerinde uzmanlaşmaya çalışır. Bu sayede, her yeni model bir önceki modelin hatalarını telafi etme görevini üstlenir.

Mesela sadece yanlış yapılanları aktarırsak o zaman ise çok az veri aktarmış oluruz.

Adım 2- Hata Oranı ve Ağırlık Hesabı

Yanlış sınıflanan satırlar : ID 6

Başlangıçta tüm örnekler eşit ağırlıkla başlar: $w_i = \frac{1}{7} \approx 0.143$

Hata oranı:

$$\epsilon = \frac{\text{Yanlış sınıflanan toplam ağırlık}}{\text{Toplam ağırlık}} = \frac{0.143}{1} = 0.143$$

Ağırlık katsayısı:

$$\alpha = \frac{1}{2} \ln \left(\frac{1 - \epsilon}{\epsilon} \right) = \frac{1}{2} \ln \left(\frac{1 - 0.143}{0.143} \right) \approx 0.895$$

- İlk hesaplamadan önce tüm row'lara aynı ağırlık verilir. Örneğin 7 row olduğu için 1/7.
- İlk hesaplama sonrasında sadece 1 hata yapıldığını görüyoruz. Buradan hata oranını belirtilen formüle ölçüyoruz ve ağırlık katsayısını 0.895 olarak buluyoruz. Bu ağırlık oranı artık bu decision tree'nin kullanacağı ağırlık katsayısı
- Bir sonraki adımda diğer DT'ye aktarılması için row'ların ağırlık katsayılarını da güncelleyeceğiz

-Hata oranına (ϵ) bakarak, bu ilk zayıf modelin ne kadar güvenilir ve önemli olduğunu gösteren bir katsayı hesaplanır.

-Formüle göre ($1/2 \ln((1 - \epsilon)/\epsilon)$), hata oranı ne kadar düşükse (ϵ ne kadar 0'a yakınsa), modelin ağırlık katsayısı (α) o kadar yüksek olur. Bu, doğru tahminler yapan modele daha fazla söz hakkı verileceği anlamına gelir.

Adım 3- Ağırlıkların Güncellenmesi

Güncelleme Formülleri

- Doğru sınıflananlar için:

$$w'_i = w_i \times e^{-\alpha} = 0.143 \times e^{-0.895} \approx 0.058$$

- Yanlış sınıflananlar için:

$$w'_i = w_i \times e^{\alpha} = 0.143 \times e^{0.895} \approx 0.354$$

Bu formüllerin temel amacı, bir önceki modelin zorlandığı örneklerle daha fazla önem verirken, kolayca doğru tahmin ettiği örneklerle daha az önem vermektir.

Doğru sınıflananlar için yukarıdaki formül, doğru tahmin edilen örneğin ağırlığını azaltmak için kullanılır.

Yanlış sınıflananlar için formül ise, yanlış tahmin edilen örneğin ağırlığını artırmak için kullanılır

Yanlış sınıflandırılan row'lar daha yüksek ağırlıklarla tekrar sınıflandırılıyor. Böylece bir sonraki DT'ye aktarılırken seçilme olasılığını çok daha yüksek yapıyoruz. Çünkü bir sonraki DT'ye aktarılırken bin aralığı denilen bir mantıkla hareket edecek ve rastgele 0-1 arasında ürettiği sayılardan seçmeye çalışacak. Fakat önce bu ağırlıkların toplamı 1 (yüzde yüz) yapmadığı için normalizasyon yapacağız.

ID	Doğru mu?	Yeni Ağırlık (w')
1	Yes	0.058
2	Yes	0.058
3	Yes	0.058
4	Yes	0.058
5	Yes	0.058
6	No	0.354
7	Yes	0.058

Table 3: Güncellenmiş ağırlıklar (Adım 3 Sonrası)

Adım 4- Normalizasyon

Toplam:

$$T = (6 \times 0.058) + 0.354 = 0.348 + 0.354 = 0.702$$

Normalleştirilmiş ağırlıklar:

$$\text{Doğru sınıflananlar: } \frac{0.058}{0.702} \approx 0.083$$

$$\text{Yanlış sınıflanan (ID 6): } \frac{0.354}{0.702} \approx 0.504$$

Normalize edilen ağırlıklara bakılarak bin aralıkları oluşturulur.

Önceki adımlarda, doğru ve yanlış sınıflandırılan örneklerin yeni ağırlıklarını hesaplamıştık. Şimdi, bu yeni ağırlıkların toplamını bulup, her birini bu toplama bölüyoruz.

Bu işlem, ağırlıkların toplamının tekrar 1 olmasını sağlamak için yapılır.

Doğru Sınıflananlar: $0.058/0.702 \approx 0.083$. Ağırlıkları büyük oranda düştü.

Yanlış Sınıflananlar: $0.354/0.702 \approx 0.504$. Ağırlığı inanılmaz derecede yükseldi.

! Bu normalleştirme işleminin sonucunda ortaya çıkan en çarpıcı şey, algoritmanın yanlış yaptığı tek örneğin ağırlığının artık tüm veri setinin yarısından fazlasına denk gelmesidir (≈ 0.504).

ID	Doğru mu?	Normalleştirilmiş Ağırlık (w'')
1	Yes	0.083
2	Yes	0.083
3	Yes	0.083
4	Yes	0.083
5	Yes	0.083
6	No	0.504
7	Yes	0.083

Table 4: Normalleştirilmiş ağırlıklar (Adım 4 Sonrası)

Adım 5- Bin Ataması

ID	Normalleştirilmiş Ağırlık	Bin Aralığı
1	0.083	0.000 - 0.083
2	0.083	0.083 - 0.166
3	0.083	0.166 - 0.249
4	0.083	0.249 - 0.332
5	0.083	0.332 - 0.415
6	0.504	0.415 - 0.919
7	0.083	0.919 - 1.000

Table 5: Bin aralıkları (Adım 5 Sonrası)

Bin Aralığı: Bu, normalleştirilmiş ağırlıkların kümülatif (birikimli) olarak toplandığı bir aralıktır.

- Örneğin, ilk 5 örnek (ID 1-5) 0.083'lük ağırlıklara sahiptir. Bu, aralığın her adımıda 0.083 artarak ilerlediği anlamına gelir.
- En kritik kısım, **ID 6'nın aralığıdır**. Ağırlığı 0.504 olduğu için, bu aralık tek başına 0.415'ten 0.919'a kadar uzanır. Bu aralığın uzunluğu, bu örneğin ne kadar yüksek bir olasılığa sahip olduğunu gösterir.

Bu "bin" sistemi, bir sonraki zayıf modelin eğitim verisini oluşturmak için kullanılır. Algoritma, 0 ile 1 arasında rastgele sayılar üretir ve bu sayılar hangi aralığa düşerse o veri noktasını seçer.

- **ID 6'nın bin aralığı çok geniş** olduğu için, üretilecek rastgele bir sayının bu aralığa düşme olasılığı çok yüksektir.

- Bu da, bir sonraki eğitim veri setinde ID 6'nın birden fazla kez yer alacağı anlamına gelir.

Böylece, algoritma yanlış yaptığı örneği sonraki modele **tekrar tekrar göstererek** o hatayı düzeltmeye zorlar. Bu mekanizma, AdaBoost'un hatalardan öğrenme ve her seferinde daha güçlü bir model oluşturma gücünü sağlayan temel adımlardan biridir.

Adım 6- İkinci Stump ve Tahminler

Simdi güncellenmiş veri kümesi ile ikinci stump seçilir. Bu seçilme esnasında 0-1 arasında rastgele sayılar oluşturulup iterasyon yapılır. Örneğin 7 kere seçilen 0-1 arasında bir değer 0.415 - 0.919 arasına düşme olasılığı diğerlerinden çok daha yüksektir. Muhtemelen 7 değer seçilirse 3-4 tanesi daha önce yanlış sınıflandırılan değerimiz olacaktır. Bin aralığında en büyük ağırlığa sahip olan ID 6 yüksek olasılıkla yeniden seçilir, bu nedenle ikinci stump ID 6 üzerine yoğunlaşabilir.

Mesela elimizde normalde 1000 data var. İkinci stump'a veri aktarırken de diyelim ki 1000 data aktaracağız. Ama bu 1000 datayı olduğu gibi direkt aktarmayacağız. Yanlış tahmin edilenleri ön plana çıkaracak şekilde olacak bu işlem.

*Örneğin ikinci stump şöyle olsun;

GPA > 3.0 mı?

Evet→ Yes Hayır→No

Bu stump'ın yeni tahminleri ve doğruluk durumu ayrı tabloda gösterilebilir.

ID	GPA	Interview Score	Approval (Y)	Tahmin	Doğru mu?
1	<3.0	Düşük	No	No	Yes
2	<3.0	Yüksek	Yes	No	No
3	<3.0	Yüksek	Yes	No	No
4	>3.0	Düşük	No	Yes	No
5	>3.0	Normal	Yes	Yes	Yes
6	>3.0	Normal	Yes	Yes	Yes
7	>3.0	Normal	Yes	Yes	Yes

Table 6: İkinci stump sonuçları: GPA >3.0 mı?

Tabloda görüldüğü gibi yanlış tahmin edilen row 3 kez seçilmiş (rastgele oluşturulan 0-1 arasındaki rakamlar sonucunda gayet olası bir sonuç) ve tekrar başka bir DT'de training'e girecektir.

Adım 7- Hata Oranı ve Ağırlık Hesabı (İkinci Stump)

Yanlış sınıflanan satırlar: ID 2, 3, 4.

Hesaplamalar

Toplam ağırlık:

$$\epsilon = \sum (\text{yanlış sınıflananların ağırlıkları}) = 0.083 + 0.083 + 0.083 = 0.249$$

Ağırlık katsayısı:

$$\alpha_2 = \frac{1}{2} \ln \left(\frac{1 - 0.249}{0.249} \right) = \frac{1}{2} \ln(3.016) \approx 0.552$$

Adım 8- Nihai Tahmin Fonksiyonu

AdaBoost nihai tahmini:

$$F(x) = \alpha_1 h_1(x) + \alpha_2 h_2(x)$$

- $\alpha_1 \approx 0.895$ (Interview Score stump),
- $\alpha_2 \approx 0.552$ (GPA stump),
- $h_m(x) = +1$ (Yes), -1 (No).

EXAMPLE: Yeni Test Verisi;

Test verisi:

- GPA = 3.2,
- Interview Score = Yüksek.

Hesaplama:

$$F(x) = (0.895)(+1) + (0.552)(+1) = 0.895 + 0.552 = 1.447$$

İlk stump tahmini:

Interview Score = Yüksek $\rightarrow$ Yes (+1).

Sonuç:

$$\text{sign}(F(x)) = +1 \rightarrow \text{Yes}$$

İkinci stump tahmini:

GPA $> 3.0 \rightarrow$ Yes (+1).

EXAMPLE: Bir Başka Test Verisi

Test verisi:

- GPA = 2.8,
- Interview Score = Yüksek.

İlk stump tahmini:

Interview Score = Yüksek $\rightarrow$ Yes (+1).

İkinci stump tahmini:

GPA > 3.0 mı? Hayır $\rightarrow$ No (-1).

Hesaplama:

$$F(x) = (0.895)(+1) + (0.552)(-1) = 0.895 - 0.552 = 0.343$$

Sonuç:

$$\text{sign}(F(x)) = +1 \rightarrow \text{Yes}$$

Burada ilk DT'nin ağırlığı daha yüksek olduğu için ikinciden No çıksa da genel sonuç Yes'e dönüyor.

16 Şema: Nihai Model Akışı

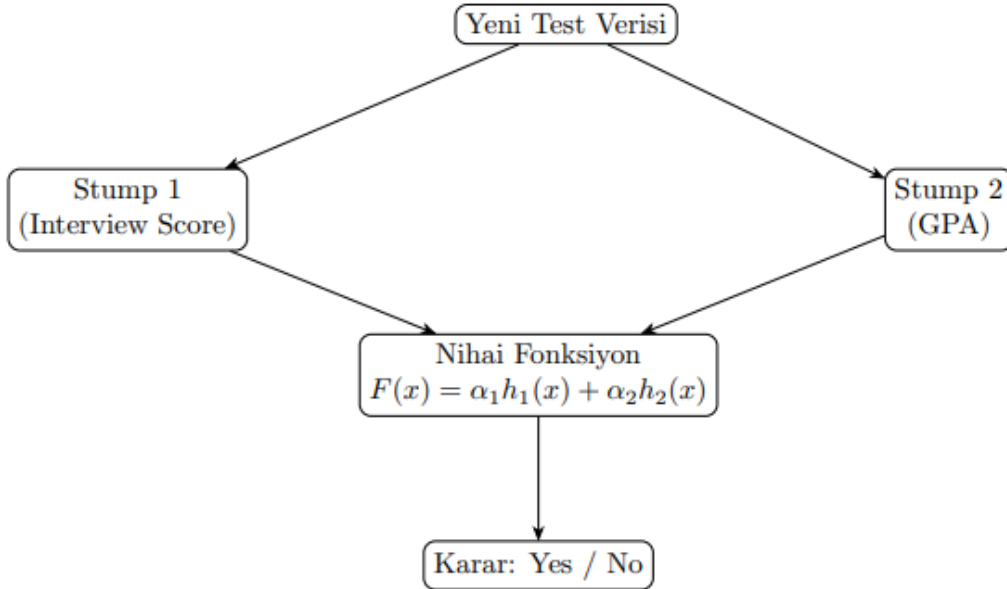


Figure 4: AdaBoost nihai karar akışı

Adaboost Regressor

AdaBoost Regressor, temel olarak zayıf regresyon modellerini (örneğin derinliği düşük karar ağaçları) ardışık olarak eğitir ve bu modellerin hatalarına göre ağırlıklandırılmış bir tahmin oluşturur.

Mantığı, sınıflandırma versiyonuyla çok benzerdir:

- Birçok zayıf regresyon modelini (örneğin, basit karar ağaçları) bir araya getirir.
- Her adımda, modelin **en büyük tahmini hataları** (gerçek değerden en uzak tahminleri) yaptığı verilere daha fazla ağırlık verir.
- Bu sayede, bir sonraki model en zorlandığı örnekleri öğrenmeye odaklanır.
- Sonuç olarak, tüm zayıf modellerin tahminlerini bir araya getirerek daha doğru bir nihai tahmin elde eder.

Hata tanımı;

Classifier'de→Yanlış Sınıflandırma Hatası: Tahmin doğru mu, yanlış mı? (İki durumlu).

Regressor'de→Rezidu (Kalıntı) Hatası: Tahmin edilen değer ile gerçek değer arasındaki farkın büyüklüğü. (Sayısal).

Ağırlık güncellemesi;

Classifier'de→Hata, yanlış veya doğru olarak ikili bir şekilde değerlendirilir. Yanlış tahmin edilenlerin ağırlığı artar.

Regressor'de→Hatanın **büyüklüğüne** göre ağırlıklar güncellenir. Hata ne kadar büyükse, o örneğin ağırlığı o kadar fazla artırılır.

Son Tahmin;

Classifier'de→Zayıf modellerin oylarının **ağırlıklı çoğunluk oylaması** ile birleştirilmesiyle bulunur.

Regressor'de→Zayıf modellerin tahminlerinin **ağırlıklı ortalaması** alınarak bulunur.

Regresyon Temel Fikir

AdaBoost Regressor'ın amacı:

$$F(x) = \sum_{m=1}^M \alpha_m h_m(x)$$

fonksiyonu ile x için bir regresyon tahmini üretmektir.

Her iterasyonda:

1. $r_i^{(m)} = y_i - F_{m-1}(x_i)$ residual (artık) değeri hesaplanır.
2. $h_m(x)$ modeli, residual değerleri tahmin etmek için eğitilir.
3. $F_m(x) = F_{m-1}(x) + \alpha_m h_m(x)$ ile model güncellenir.

Not: Eğer $\alpha_m = 1$ alınırsa, bu durumda model direkt olarak her yeni öğreniciyi $F(x)$ fonksiyonuna ekler. Bazı implementasyonlarda bu katsayı hata oranlarına göre öğrenilir.

18 Örnek Veri Seti

ID	Experience (Years)	Salary (k\$)
1	1	40
2	2	45
3	3	50
4	4	58
5	5	60
6	6	65

Table 7: AdaBoost Regressor için örnek veri seti

19 Adım 1: İlk Model ve Residual Hesabı

İlk model genellikle basit bir karar ağacı veya sabit bir tahmin olabilir. Örneğin:

$$F_0(x) = \text{Ortalama Salary} = \frac{40 + 45 + 50 + 58 + 60 + 65}{6} = 53$$

Gerçek Y	Tahmin $F_0(x)$	Residual ($r_1 = Y - F_0(x)$)
40	53	-13
45	53	-8
50	53	-3
58	53	+5
60	53	+7
65	53	+12

Table 8: İlk model tahminleri ve residual hesapları

20 Adım 2: Residual'lara Uygun Yeni Model

Yeni model $h_1(x)$, residual değerlerini tahmin etmeye çalışır. Basit bir karar ağacı, residual'lara göre bölünerek aşağıdaki gibi olabilir:

$$h_1(x) = \begin{cases} -10 & \text{if Experience} < 3 \\ +8 & \text{otherwise} \end{cases}$$

Güncellenmiş model:

$$F_1(x) = F_0(x) + \alpha_1 h_1(x)$$

Burada α_1 genellikle 1 alınır, ya da optimize edilebilir.

21 Adım 3: Yeni Residual ve İterasyon Devamı

Yeni residual'lar:

$$r_2 = y_i - F_1(x_i)$$

Her iterasyonda bu adımlar devam eder. Amaç, artıkların sıfıra yaklaşmasıdır. Yani model artık hataları öğrenmeyi bırakana kadar eğitim devam eder.

22 Sonuç

AdaBoost Regressor:

- Zayıf regresyon modellerinin artık hatalarını öğrenerek güçlü hale gelir.
- Her adımda residual'ları hedef olarak alır.
- Sınıflandırmaya göre daha yumuşak ve sayısal çıktılar üretir.

Boosting algoritmalarının regresyon versiyonları genellikle *robust* ve *ensemble* yöntemler olarak kullanılır. AdaBoost Regressor, özellikle veri setindeki gürültülere duyarlıdır, bu nedenle genellikle **gradient boosting** gibi yöntemlere kıyasla daha az tercih edilse de eğitim açısından faydalı bir yapı sunar.